

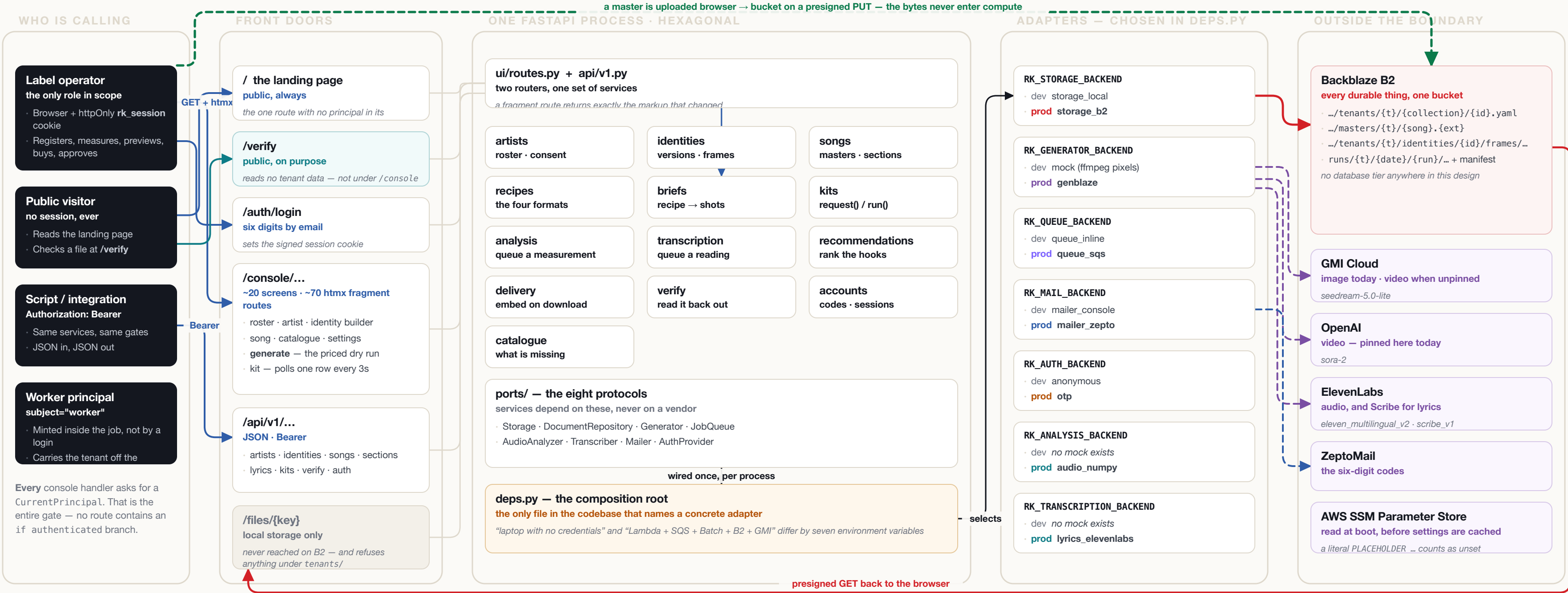
System architecture, and every trace a user leaves.

A record label registers an artist, builds that artist's identity once, attaches songs, measures each master, generates content designed to be imitated, approves it, and hands it off with the disclosure travelling inside the file.

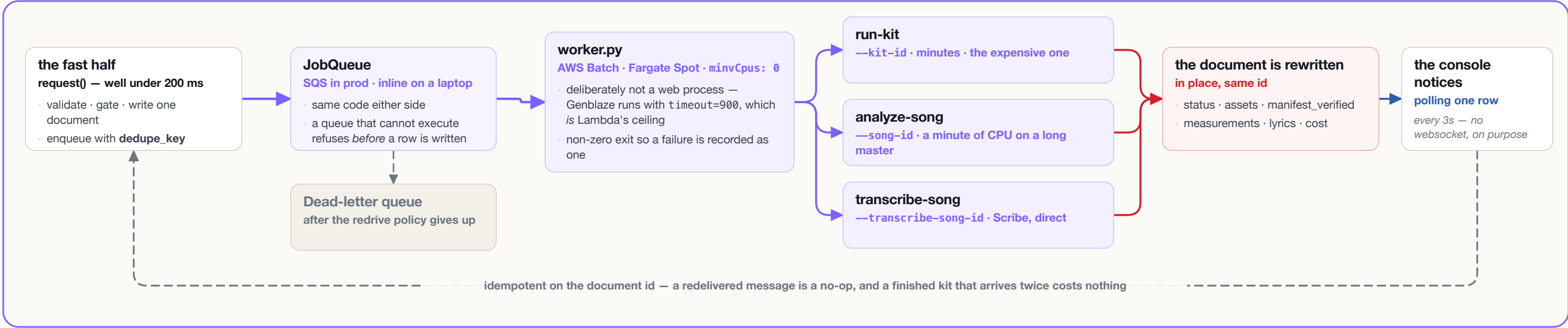
This document maps that end to end: every surface a person touches, every document and object the touch writes, every claim the system makes about *how* it knows something, and the exact boundary of what taking it back can reach.

- request / response** the ordinary synchronous path
- durable bytes** everything that outlives the process
- queued work** never inside an HTTP request
- browser ↔ bucket** presigned; the bytes never enter compute
- a vendor call** billed — and observably billed on failure too
- provenance** the claim that travels inside the file
- a refusal** on purpose, in the service layer, with a test

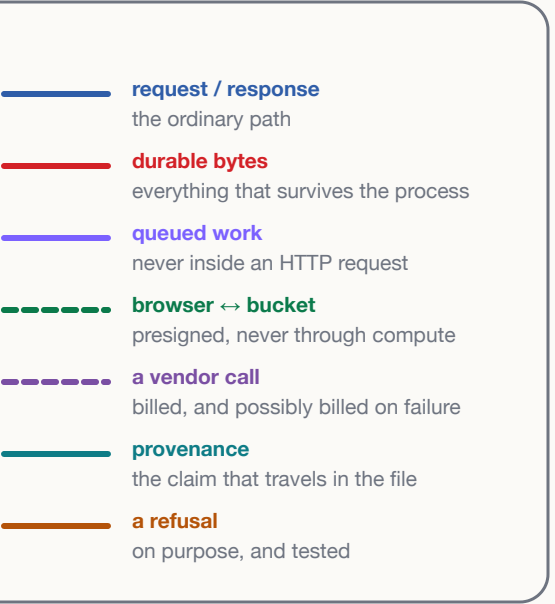
- 01 The whole system on one sheet**
Actors, front doors, thirteen services, eight ports, seven adapter axes, one bucket — and the asynchronous lane everything slow is pushed into.
- 02 The journey · onboarding, and the master**
Nine touchpoints from the sign-in code to the hook window, each with what it writes, what it records about how, and what undoing it reaches.
- 03 The journey · buying a run, and what leaves**
Nine more, from the priced dry run to deletion. Two of them write nothing at all; one of them is where every dollar goes.
- 04 Residual actions — the ledger**
The whole key space, every durable record, what each claim rests on, and the exact limits of every undo.
- 05 The expensive path**
One resolution function with two callers, the two-stage identity lock, the still index, and every refusal a plan screen can raise.
- 06 Deployment**
Lambda, SQS, Batch on Fargate Spot, SSM and B2 — against the same code running on a laptop with no credentials.
- 07 The trust argument**
The provenance loop, the fifteen refusals in the product, and what is deliberately absent.



THE ASYNCHRONOUS HALF — NOTHING SLOW HAPPENS IN A REQUEST

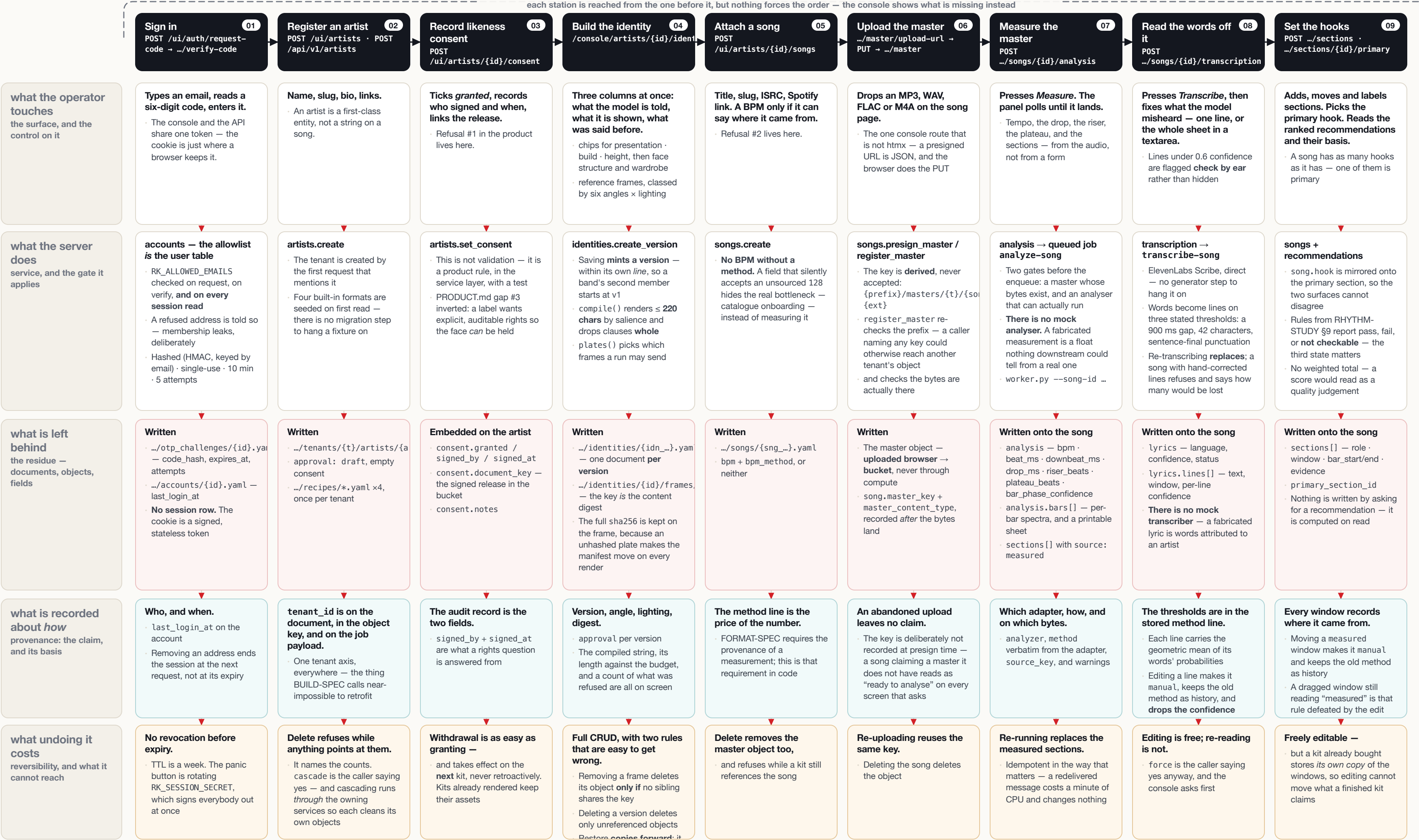


HOW TO READ THE WIRES



Onboarding, and everything the master is asked

Nine touchpoints. Read down a column for one action and everything it leaves behind; read across a lane to compare what the system keeps.





Everything a user leaves behind, and what it costs to take back

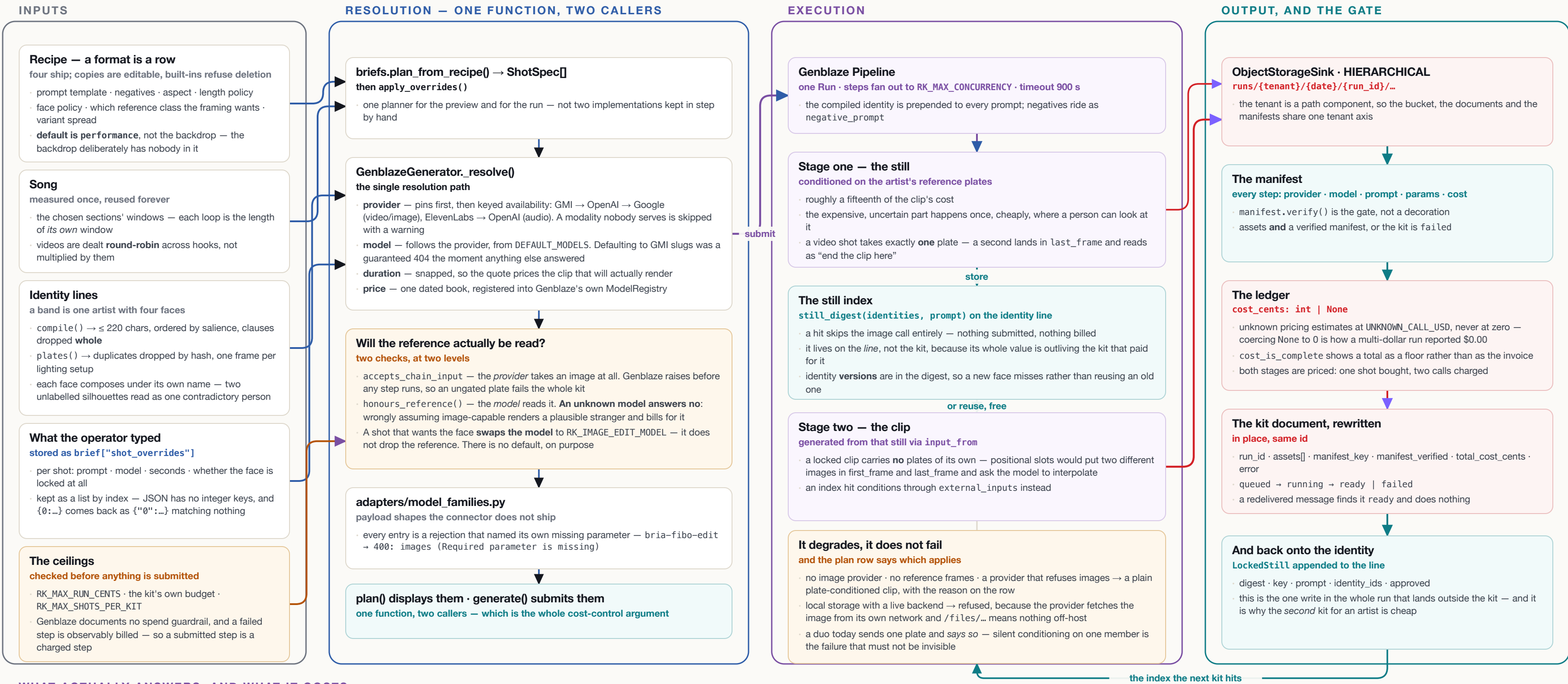
The product keeps no event log. Every claim it makes is a field on a document beside the sentence explaining where the field came from — so this page is the audit trail.

ONE BUCKET · THE WHOLE KEY SPACE



One resolution function, and the two-stage lock around a face

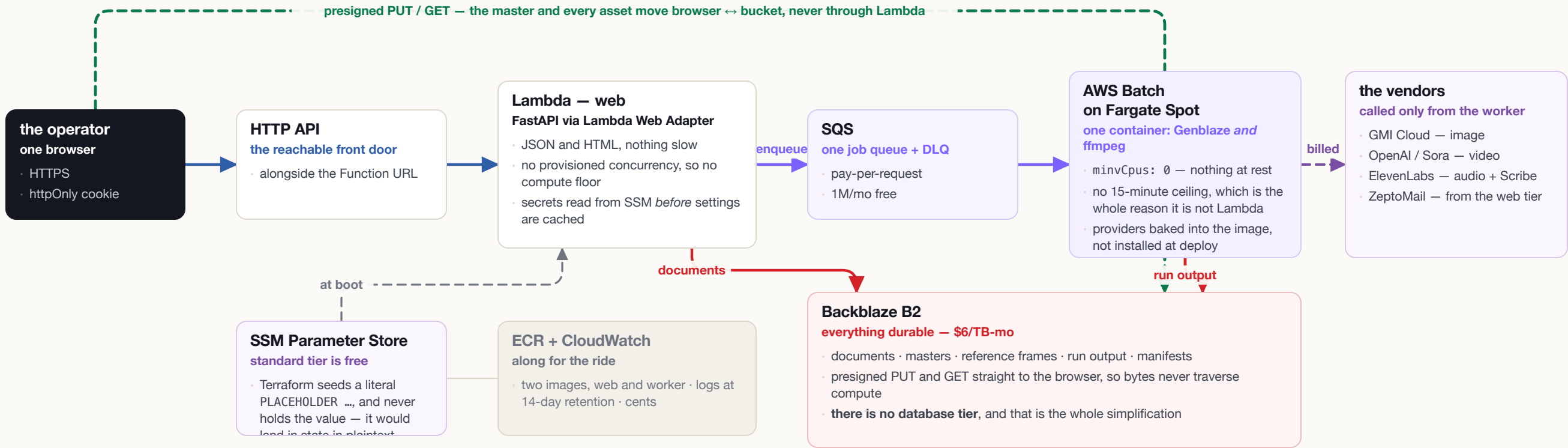
Everything between a brief and a billed provider call. The load-bearing property is that the dry run and the real run are the same function — so the payload on screen is the payload on the wire.



Five services, one durable store, and no database tier

The same code runs on a laptop with zero credentials and on Lambda + SQS + Batch against B2 and three vendors. Seven environment variables are the entire difference.

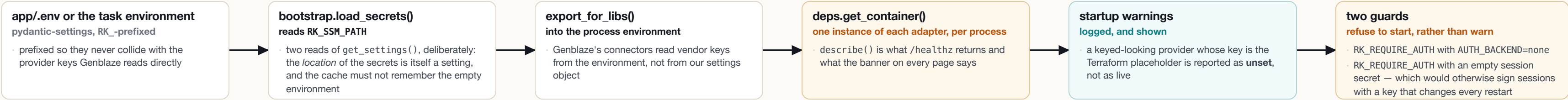
PRODUCTION — AWS RUNS THE LOGIC, B2 HOLDS THE BYTES



Realistic idle cost: under \$1/month — B2 storage and a container image. There is no compute floor anywhere, and the reason is that the relational schema this product would otherwise need exists almost entirely to serve fans: attribution joins, leaderboards, click ledgers, reward rows. Those are deferred, and the pressure for a database tier goes with them. What the label actually stores is documents, and a bucket is excellent at documents.

What “mock” does not mean. Real: the Pipeline, the sink, the key layout, content hashing, the manifest, manifest.verify(), the cost ledger and embed-on-delivery. Synthetic: the pixels, which ffmpeg draws, and the unit costs, which come from a table. So the manifest verified on a laptop is produced by the same code that produces the one in B2 — which is the only reason a zero-credential demo is worth having.

BOOT, CONFIGURATION, AND THE TWO GUARDS THAT REFUSE TO START



TURNING AUTHENTICATION ON	WHY IT IS SHAPED THIS WAY	WHAT ADDING AUTH COST	BECAUSE
RK_AUTH_BACKEND=otp	the default none is a complete implementation that admits everyone, so a fresh checkout has no login to get past	one provider, one service, one page, one exception handler	every request already resolved a Principal, and every document, object key and job payload already carried its tenant_id
RK_ALLOWED_EMAILS=...	this is the user table. No registration. Checked on request, on verify, and on every session read	no service changed	services take ports; routes take services; only deps.py names an adapter
RK_SESSION_SECRET=...	sessions are a signed token, not a row — so there is no revocation before expiry, and rotating this is the panic button	no template and no storage key changed shape	the tenant axis was already in the path — “componentized, and we add auth later” became a configuration change rather than a refactor
RK_MAIL_BACKEND=zeptomail	with no mailer the code goes to the log, and back to the caller only in RK_ENV=dev — either condition alone would be a bypass	no route grew an if authenticated branch	one AuthError handler decides what a failure looks like: a browser is redirected with ?next= preserved, an htmlx request gets HX-Redirect, a script gets 401 JSON

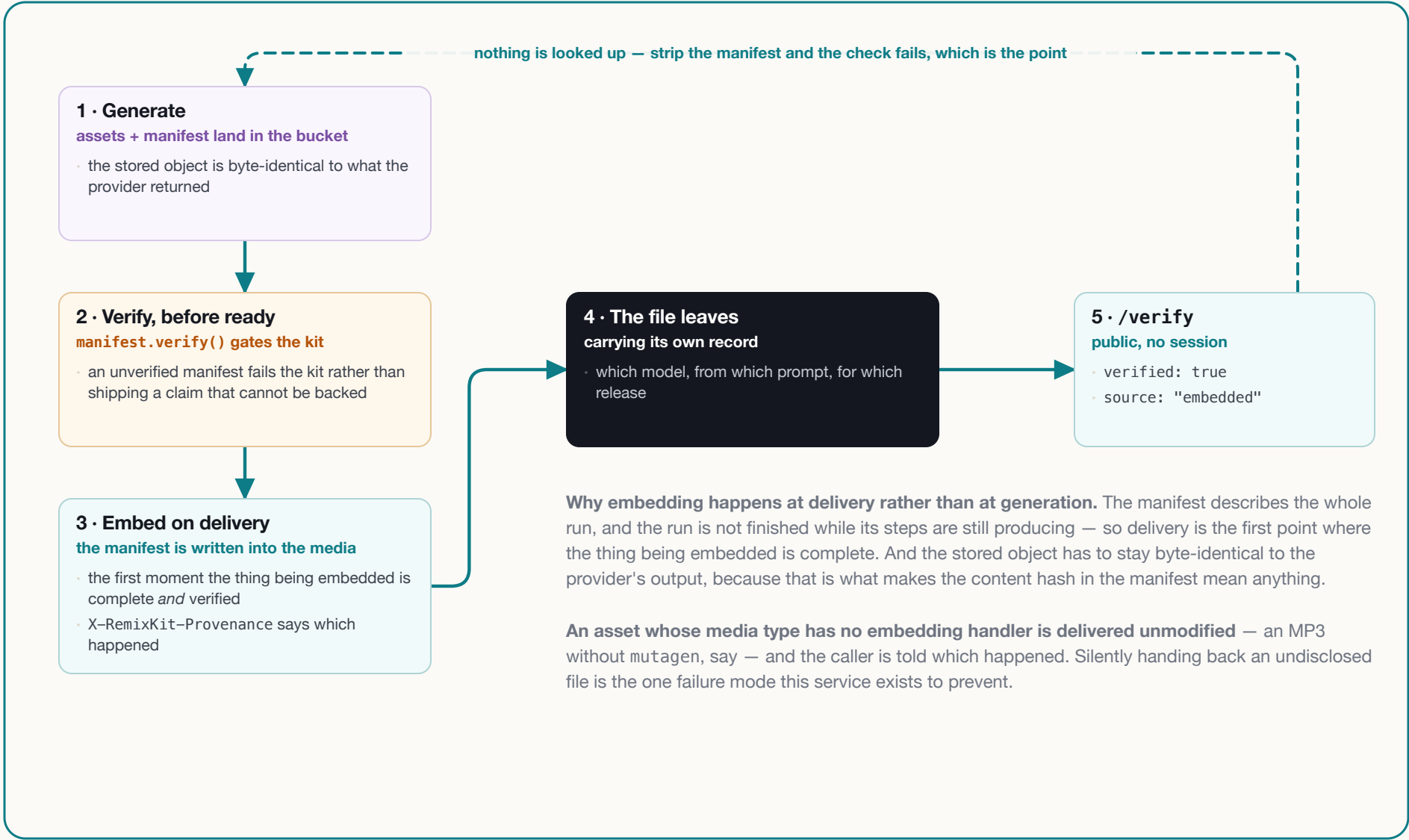
THE LAPTOP, WITH NO CREDENTIALS

AXIS	DEV	PRODUCTION
RK_STORAGE_BACKEND	local	b2
RK_GENERATOR_BACKEND	mock	genblaze
RK_QUEUE_BACKEND	inline	sqs
RK_MAIL_BACKEND	console	zeptomail
RK_AUTH_BACKEND	none	otp
RK_ANALYSIS_BACKEND	refuses	auto (numpy)
RK_TRANSCRIPTION_BACKEND	refuses	auto (Scribe)

Not one line of application code differs between the two columns — services take ports, and only deps.py names an adapter.

Two axes have no mock, deliberately. A mocked measurement is a float nothing downstream could tell from a real one; a mocked lyric is words attributed to an artist. Both refuse and name what is missing — on the song page and in /healthz.

THE PROVENANCE LOOP — TESTED AS A LOOP, NOT AS THREE ASSERTIONS



WHAT IS DELIBERATELY NOT HERE

ABSENT	WHY
attribution links · /r/{code} · the disclosure gate · leaderboards · rewards · composite sessions · K-factor	all of it serves the <i>fan</i> role, which this scope defers. They are also what would force a relational database — their absence is why there is no database tier
a websocket	the kit page polls one row every three seconds; a viral release exercises storage reads, not sockets
a CDN	there is no read path hot enough to justify one at this scope
a “which archetype goes viral” recommender	research calls that folklore. The song page ranks hooks against stated rules and reports not checkable when it cannot — there is no weighted total anywhere, because a single score would read as a quality judgement the material cannot support
an event log or audit trail	provenance is a field on the document beside the value it describes. That is a deliberate trade: it answers “how was this measured” perfectly and “who changed it last Tuesday” not at all

EVERY REFUSAL IN THE PRODUCT, AND WHAT IT IS PROTECTING

#	THE REFUSAL	PROTECTING
01	No recorded likeness consent → no kit	a kit holds the artist's face invariant across every shot, and that needs auditable rights. Withdrawal takes effect on the next kit, not retroactively
02	A BPM with no method → refused	a field that silently accepts an unsourced 128 hides catalogue onboarding, which is the actual bottleneck
03	Estimate over RK_MAX_RUN_CENTS or the kit budget → refused before submission	Genblaze documents no spend guardrail and a failed step is observably billed, so the only safe assumption is that a submitted step is a charged step
04	A queue that cannot execute → refused before a row is written	otherwise a kit sits at queued forever with nothing coming for it
05	Assets without a verified manifest → the kit fails	the disclosure argument <i>is</i> the product; shipping an unbacked claim is worse than shipping nothing
06	No numpy → the analyser refuses and names what is missing	a mocked measurement is a float nothing downstream could tell from a real one
07	No credentialed transcriber → refused	a fabricated lyric is words attributed to an artist, offered to the generate form and burned into a clip
08	Re-transcribing over hand-corrected lines → refused unless forced	a lyric is one continuous reading; interleaving old corrections with new lines produces a transcript that is neither
09	Deleting an artist with dependents → refused, with the counts named	silent orphaning is the worst of the three options — a roster that looks clean while the catalogue counts songs nobody can open
10	A live generator on local storage → the plate is refused	the provider fetches the image from its own network, and /files/... means nothing off-host. Sending a dead link would look like it worked
11	An unknown image model → treated as not reading references	the safe direction: a model wrongly treated as image-capable renders a plausible stranger and bills full price for it
12	RK_IMAGE_EDIT_MODEL unset → the plan says no likeness will be held	there is no default because a hardcoded slug 404'd once already, and GMI ships no catalog endpoint to check one against
13	A master key that is not this song's → refused	it arrives from a browser; a caller who could name any key could point a song at another tenant's object
14	An email outside the allowlist → told so	this leaks membership, deliberately: with a handful of known colleagues the real failure is somebody mistyping their own address
15	RK_REQUIRE_AUTH without a backend or a secret → the process refuses to start	a warning here would mean a deployment that looks authenticated and is not